Project Scoping Submission

# SkinSort

IE7374 Machine Learning Operations

**Team 18**

Northeastern University

## Team Members


Bala Asrith Chavala

Dheepak Karan Elumalai Santhakumari

Elenta Suzan Jacob

Karuniya Premnath

Rahul Gudivada

Sai Dheekshit Reddy Antham

# 1. Introduction

Most people have spots on their skin: moles, freckles, small bumps or rough patches. Doctors call any unusual spot or patch of skin a **lesion**. The word sounds alarming, but it simply means an area of skin that looks different from the skin around it. The vast majority of lesions are harmless, like an ordinary mole. A small number, however, are skin cancers.

The most dangerous of these is **melanoma**, a cancer that starts in the cells that give skin its colour. Melanoma often looks like a new mole, or an existing mole that changes in size, shape or colour. When it is caught early it is usually very treatable, but if it is left too long it can spread to other parts of the body. Other skin cancers, such as squamous cell carcinoma and basal cell carcinoma, are more common and grow more slowly, but they still need a doctor to look at them.

The problem is time. Seeing a dermatologist (a skin specialist) in the US can take weeks or months, and cases are usually looked at in the order they arrive. That means a possible melanoma can sit in the same line as dozens of harmless moles.

**SkinSort** is designed to fix the order of that line. A patient takes a photo of the spot with their **own smartphone camera**, uploads it through a simple web page, and answers four quick questions: how long they have had it, whether it has changed, whether it itches, hurts or bleeds, and whether there are other spots like it. Before the photo is accepted, the app checks that it is not blurry or too dark and asks for a retake if needed. A machine-learning model then looks at the photo and the answers together and decides how urgent the case looks: **routine**, **priority** or **urgent**.

That urgency score is used to sort a review list that only the dermatologist sees. The most worrying cases rise to the top, while cases that have been waiting longer also move up gradually, so nobody is forgotten. The patient never sees a score or a diagnosis, and a dermatologist still looks at every single case. In short, SkinSort does not replace the doctor. It helps the doctor decide which case to open next.

For this course, the focus is on building the full machine-learning operations (MLOps) pipeline around this idea: versioned data, tracked experiments, a model served in the cloud, automated testing and deployment, and monitoring that tells us when the model needs to be improved.

# 2. Dataset Information

## 2.1 Dataset Introduction

No single public dataset has everything we need, so we combine four. Two give us a large number of labelled skin-spot images, one gives us real smartphone photos with patient answers, and one lets us check that the model works fairly across different skin tones.

| Dataset | Approx. size | Image type | Role in project |
|---|---|---|---|
| ISIC Archive | ~70,000 images (subset used) | Dermoscopic (taken with a special skin magnifier) | Main training data for recognising lesions |
| HAM10000 | 10,015 images | Dermoscopic | Extra training data; much of it is already in ISIC, so the two are merged and duplicates removed |

| Dataset | Approx. size | Image type | Role in project |
|---|---|---|---|
| PAD-UFES-20 | ~2,300 images | Smartphone photos | Real phone photos plus patient answers (itch, bleeding, growth). Closest match to how our users will submit |
| Fitzpatrick17k | ~16,500 images | Clinical photos | Used only to test fairness across skin tones |

The images in each dataset come with a diagnosis (for example, melanoma or harmless mole). We convert each diagnosis into one of our three urgency levels, as described in Section 3.

## 2.2 Data Card

| Field | ISIC Archive | HAM10000 | PAD-UFES-20 | Fitzpatrick17k |
|---|---|---|---|---|
| Size | ~70,000 images | 10,015 images | ~2,300 images (~1,600 lesions) | ~16,500 images |
| Format | JPEG + CSV | JPEG + CSV | PNG + CSV | CSV of image links + labels |
| Image type | Dermoscopic | Dermoscopic | Smartphone | Clinical |
| Labels | Diagnosis | 7 diagnosis classes | 6 diagnosis classes | 114 skin conditions |
| Patient info | Age, sex, body site | Age, sex, body site | Age, sex, body site, skin type (partial) | Skin type (Fitzpatrick I-VI) |
| Data types | Images, categorical, numeric | Images, categorical, numeric | Images, categorical, numeric, yes/no | Images, categorical |
| License | CC-0 / CC-BY / CC-BY-NC (per image) | CC BY-NC 4.0 | CC BY 4.0 | Non-commercial research |

## 2.3 Data Sources

| Dataset | Where it comes from |
|---|---|
| ISIC Archive | `https://www.isic-archive.com` (website and public API) |
| HAM10000 | Harvard Dataverse, doi:10.7910/DVN/DBW86T |
| PAD-UFES-20 | Mendeley Data, doi:10.17632/zr7vgbcyr2.1 |
| Fitzpatrick17k | `https://github.com/mattgroh/fitzpatrick17k` |

All four are downloaded by scripts in our repository and stored in a Google Cloud Storage bucket, with versions tracked by DVC so every teammate works with exactly the same data.

## 2.4 Data Rights and Privacy

All four datasets are publicly released for research, and the people in them were de-identified by the original institutions before release. We do not try to identify anyone, and we do not collect any real patient data. Any “patient” in our demo uses images from these public datasets with made-up form

answers.

Some datasets (HAM10000 and part of ISIC) only allow non-commercial use, so SkinSort is strictly an academic project. We will credit every dataset in our repository and final report, and we will not redistribute Fitzpatrick17k images, since they are hosted on third-party websites.

HIPAA, the US law that protects patient health information, applies to hospitals and companies handling real patient records. Because we only use public, de-identified data, HIPAA does not apply to our project, and we will not claim to be HIPAA-compliant. If SkinSort were ever used with real patients, it would need encryption, strict access control, audit logs and a formal agreement with the cloud provider. Some of the data also comes from Brazil and Europe, where privacy laws such as LGPD and GDPR apply; this was handled by the original data publishers, and we only use their public release.

## 3. Data Planning and Splits

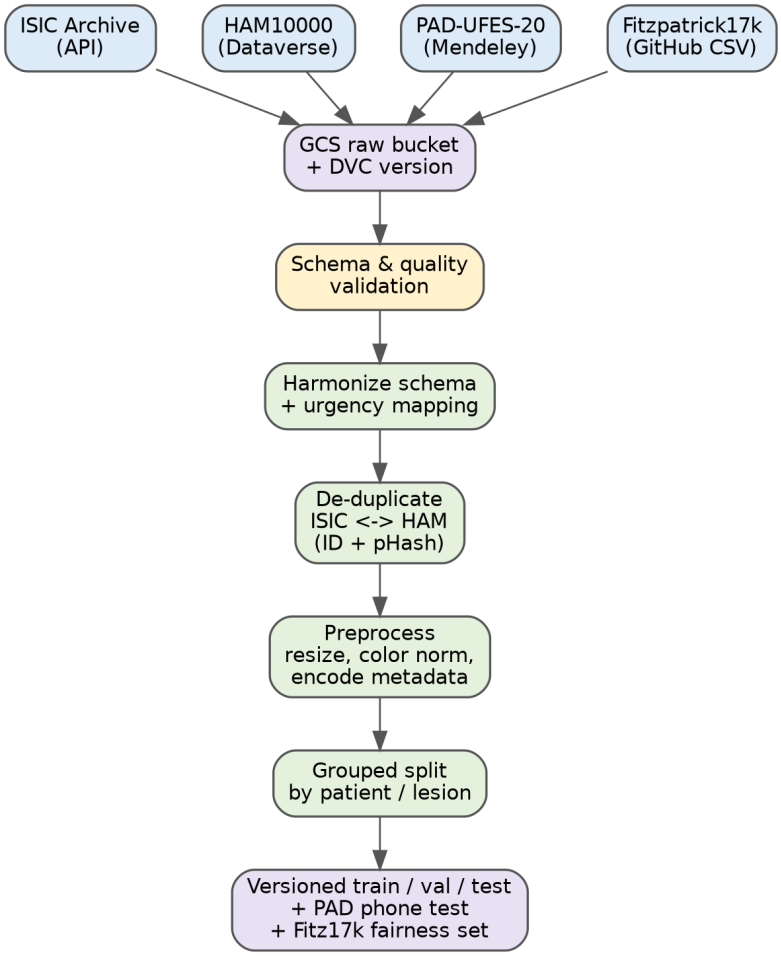



*Figure 1. How data flows from the original sources to training-ready sets*

### 3.1 Loading the Data

Each dataset is downloaded by its own script, which also records a checksum so we can tell if a file was corrupted or changed. The raw files are stored in Google Cloud Storage and tracked by DVC (think of it as Git for data). Before anything else runs, an automatic check confirms that the files have the columns we expect, the images open properly and the labels make sense. If anything looks wrong, the pipeline stops instead of quietly training on bad data.

### 3.2 Cleaning and Preparing the Data

Each dataset names things differently, so first we put everything into one common format. Next, we turn every diagnosis into an urgency level using the table below. This table is the most important labelling decision in the project, so it is saved as a file in the repository and every model records which version it was trained on.

| Diagnosis | Urgency level | Why |
|---|---|---|
| Melanoma | Urgent | Most dangerous skin cancer; delay changes the outcome |
| Squamous cell carcinoma | Urgent | Can spread; needs prompt review |
| Basal cell carcinoma | Priority | Cancer, but slow-growing and rarely spreads |
| Actinic keratosis | Priority | Can turn into cancer; should be checked soon |
| Moles and other harmless spots | Routine | Benign; can wait |

> This mapping is a draft. We will check it against a published clinical source before the final submission.

We then remove duplicates. Many HAM10000 images are also in the ISIC Archive, and if the same photo ended up in both the training and testing sets the model would look better than it really is. We match images by ID and by a visual “fingerprint” to catch copies. We also drop photos that are too blurry or too dark, resize every image to the same size, and balance the colours so that differences in lighting or camera matter less. During training we also slightly blur, crop and adjust the brightness of images so that the model gets used to the lower-quality photos people take on phones.

### 3.3 Dealing With Missing Patient Answers

Our model is meant to use both the photo and the patient’s answers. The catch is that only one dataset, PAD-UFES-20, actually has those answers. The other datasets just have the photo plus age, sex and where on the body the spot is.

So we train in two steps. First, we teach the model to recognise skin spots using the large image-only datasets, because that is where most of the examples are. Second, we teach it to also use the patient’s answers, using PAD-UFES-20. When an image does not come with answers, we simply tell the model “this information is missing” rather than making anything up. That way the model works whether or not a patient answers every question.

Because the answers come from a fairly small dataset, we will be honest about this limitation. We will also compare our model with a photo-only version, to see whether the answers genuinely help.

### 3.4 Splitting the Data

We divide the data into three parts: one to train the model, one to fine-tune it, and one kept aside for the final test. For the large image pool this is roughly 70%, 15% and 15%. The key rule is that we split by **person or lesion, not by photo**. Some datasets contain several photos of the same spot, and if one photo went into training and another into testing, the model would effectively have seen the answer already.

PAD-UFES-20 is split the same way, and its test portion is especially important because it is made of real phone photos, just like our users will send. Fitzpatrick17k is never used for training; it is kept completely aside so we can fairly measure how the model performs on lighter and darker skin. Because urgent cases are much rarer than routine ones, we give them extra weight during training so the model does not simply learn to call everything routine.

### 3.5 Managing Data

Managing data means keeping it stored safely, knowing exactly which version we are using, and being able to rebuild or update it at any time. Here is how we will do that.

**Where the data lives.** All images and data files are kept in one Google Cloud Storage bucket, organised into three areas: raw (exactly as downloaded), processed (cleaned and merged) and splits (the final training, tuning and test sets). The GitHub repository only holds the code and small pointer files, not the images themselves, which keeps the repository light and fast.

**Keeping track of versions.** We use DVC, which works like Git but for data. Every time the data changes, for example after we fix a label or remove more duplicates, DVC saves a new version and records it in GitHub. Any teammate can run one command (dvc pull) to get exactly the same data as everyone else, and we can always go back to an older version if something goes wrong.

**One repeatable pipeline.** All the steps in this section, from downloading to splitting, are written as a single pipeline. If anything changes, one command (dvc repro) re-runs only the steps that are affected. This avoids “it works on my laptop” problems and means the data is never edited by hand.

**Knowing what each model was trained on.** Whenever we train a model, MLflow automatically records the version of the code, the version of the data and the version of the urgency table. If a model ever behaves strangely, we can trace exactly what it learned from.

**Checks before use.** Automatic data checks run before every training job and on every code change. They confirm the columns, image files, labels and splits are correct, and that no photo or lesion appears in both training and test data.

**Adding new data over time.** Once SkinSort is running, new data arrives in two forms: new cases submitted through the app, and doctor corrections (simulated in this course). These are saved in the database and copied into the storage bucket on a regular schedule, then added as a new data version. When the model is retrained, it uses this updated version, while the test sets stay fixed so old and new models can be compared fairly.

**Access and safety.** Only team members’ accounts and the project’s service accounts can read or write the bucket. Cloud Storage encrypts data automatically, and versioning means accidentally deleted files can be restored.

# 4. GitHub Repository

**Link:** `https://github.com/DHEEKSHIT11/skinSort`

## 4.1 Folder Structure

```
skinsort/
|-- .github/workflows/
|-- configs/
|-- data/
|   |-- ingest/
|   |-- raw.dvc
|   `-- processed.dvc
|-- src/
|   |-- data/
|   |-- models/
|   |-- training/
|   |-- queue/
|   |-- intake/
|   `-- serving/
|-- app/
|-- monitoring/
|-- notebooks/
|-- tests/
|-- docker/
|-- dvc.yaml
|-- docker-compose.yml
|-- requirements.txt
`-- README.md
```

### What each folder is for

- **.github**/**workflows**/: automatic checks that run every time someone pushes code. They test the code, check the data and build the app so broken changes are caught early.
- **configs**/: settings files kept in one place, such as model settings, the urgency table, and the numbers that control how the review list is sorted. Changing a setting never means editing code.
- **data**/: everything about getting the data. The **ingest**/ subfolder holds the download scripts for each dataset. The **raw.dvc** and **processed.dvc** files are small pointers that tell DVC which version of the data to fetch from cloud storage; the actual images are not stored in GitHub.
- **src**/: the main project code, split by job.
  - **src**/**data**/: cleaning, merging, removing duplicates and splitting the data.
  - **src**/**models**/: the model itself, including the part that reads the photo and the part that reads the patient answers.
  - **src**/**training**/: training and testing the model, including the fairness checks, and recording results.
  - **src**/**queue**/: the logic that sorts cases for the dermatologist and records when a doctor disagrees with the model.
  - **src**/**intake**/: the patient form and the four questions, plus the optional chat assistant.

  - **src/serving/**: the API that receives a new case, runs the model and returns the urgency level.
- **app/**: the two web screens: the patient upload page and the dermatologist review dashboard.
- **monitoring/**: settings for the dashboards and alerts that watch the system once it is running.
- **notebooks/**: exploratory analysis and experiments; not used by the live system.
- **tests/**: automated tests that make sure each piece works on its own and together.
- **docker/**: recipes for packaging the app and model so they run the same on any machine or in the cloud.
- **dvc.yaml**: the step-by-step data and training pipeline, so it can be rebuilt with one command.
- **docker-compose.yml**: starts the whole system (app, API, monitoring) on a laptop for testing.
- **requirements.txt**: the Python libraries the project needs.
- **README.md**: setup and usage instructions (will be written in the GitHub repository).

# 5. Project Scope

## 5.1 Problems

The core problem is that people with potentially serious skin spots wait too long to be seen. Dermatologists are in short supply, and waits of several weeks or months are common. On top of that, cases are usually reviewed in the order they arrive, so the system has no way of knowing that the tenth case in line might be a melanoma while the first nine are harmless moles. Every day a dangerous case waits behind routine ones is a day lost.

There are also problems with the information doctors receive. Photos are often blurry or badly lit, and the patient's history (how long the spot has been there, whether it has changed) is frequently missing, so doctors spend time chasing details. Finally, many existing skin-analysis tools were trained mostly on lighter skin and on photos taken with professional equipment. They may work less well for people with darker skin, or for the ordinary phone photos that patients actually take. And when a doctor disagrees with an AI tool, that correction is rarely recorded, so the tool never learns from its mistakes.

## 5.2 Current Solutions

Today, most clinics simply review cases first-come, first-served. In some places a nurse or dermatologist skims incoming referrals to pull out the worrying ones, which helps, but it uses up the very specialist time that is already scarce, and it is inconsistent from person to person. Some health systems let the referring doctor tick an "urgent" box, but that depends on the judgement of a non-specialist, so serious cases are sometimes missed and harmless ones are sometimes fast-tracked.

There are also AI tools. Consumer apps such as SkinVision give patients a risk score directly on their phone, but that puts a medical judgement in the patient's hands, and SkinVision is not available in the US. Clinical tools such as Skin Analytics DERM (used in the UK's NHS) and DermaSensor (cleared by the FDA in 2024) are used inside clinics, but they are proprietary and often depend on special equipment rather than everyday phone photos.

### 5.3 Proposed Solution

SkinSort sits in the gap between these options. Patients use their own phone to send a photo and answer the same four short questions, so every case arrives with consistent, useful information. A model looks at the photo and the answers together and estimates how urgent the case is. If there is even a reasonable chance a case is urgent, it is moved up to the urgent group, because sending a harmless case to the doctor a little early is far better than leaving a dangerous one at the back of the line.

The dermatologist sees a review list sorted by urgency. Cases that have been waiting a long time slowly rise, so routine cases are never stuck forever. The patient never sees a score, which keeps medical judgement with the doctor, and the doctor reviews every case. Whenever the doctor disagrees with the model, that disagreement is recorded and used to improve the next version. Behind the scenes, the full MLOps pipeline keeps the data versioned, the model tested and monitored, and any new model checked for fairness across skin tones before it goes live.

## 6. Current Approach Flow Chart and Bottleneck Detection

### 6.1 Current Process

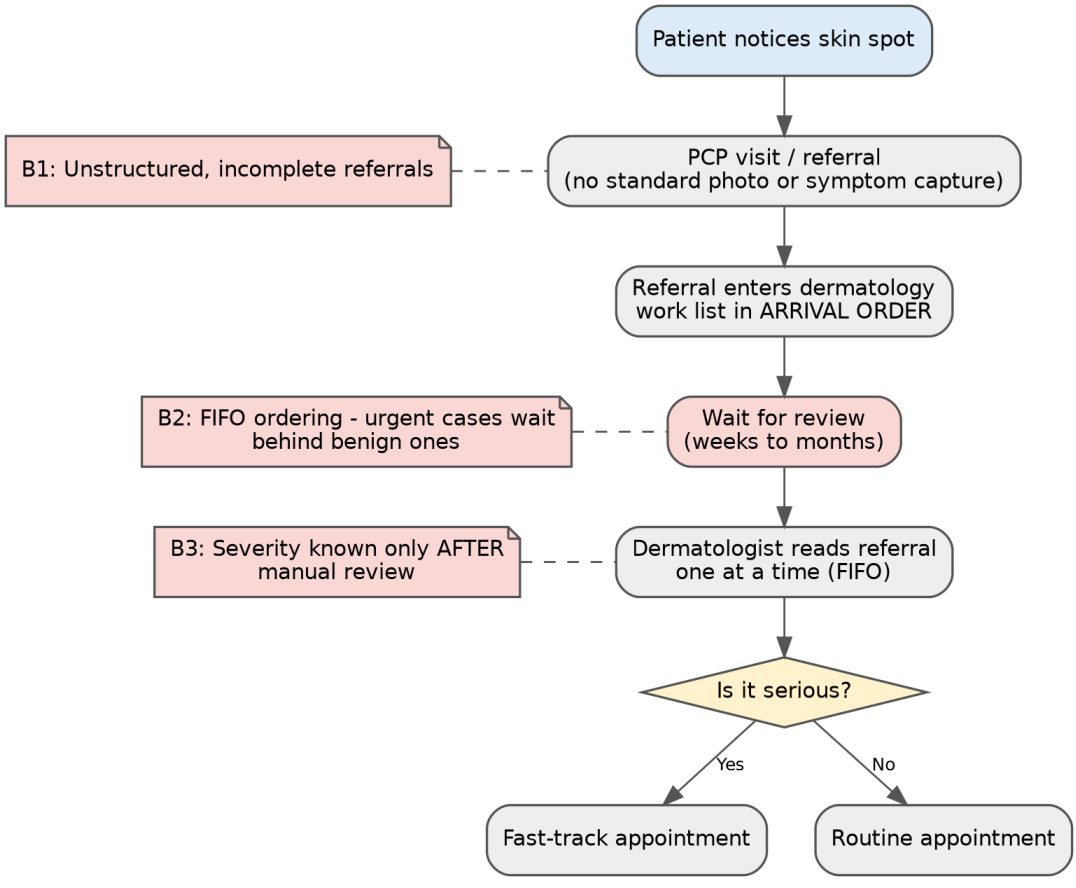



*Figure 2. How skin cases are reviewed today, with bottlenecks marked*

## 6.2 Bottlenecks and How We Improve Them

**B1: Incomplete information.** Referrals arrive with poor photos and missing history, so doctors waste time asking for more. SkinSort checks photo quality at upload and asks every patient the same four questions, so each case arrives complete.

**B2: First-come, first-served order.** An urgent case waits just as long as a harmless one. SkinSort sorts the review list by urgency, while still letting long-waiting cases rise over time.

**B3: Severity is only known after review.** The doctor has to open cases one by one to find the few that are serious. SkinSort estimates urgency the moment a case arrives, so the doctor starts with the cases most likely to need attention.

**B4: No learning from mistakes.** Doctor corrections are not captured today. SkinSort logs every disagreement and uses it to retrain and improve the model.

Our own pipeline will also have bottlenecks. Running the model is the slowest part of handling a new case, so we will optimise the model for fast predictions. Merging and cleaning four datasets is a heavy one-time job, which we will run once and save. The small amount of patient-answer data limits how much the model can learn from symptoms, which we address with the two-step training described in Section 3.3.

## 6.3 Proposed Flow

Figure 3 shows how a case moves through SkinSort. It happens in three stages.

**Stage 1: The patient.** The patient opens the SkinSort web page on their phone and fills in a short form (name, age, sex, contact details and whether they are an existing patient). They take a photo of the skin spot with their phone camera and upload it. The app immediately checks whether the photo is clear enough; if it is blurry or too dark, the patient is asked to take it again. The patient then answers the four fixed questions about how long they have had the spot, whether it has changed, whether it itches, hurts or bleeds, and whether there are other spots like it. A short summary is shown for the patient to confirm. At this point the patient's part is finished. They are told their case has been received, but they never see a score, a position in line or a diagnosis.

**Stage 2: The system (automatic).** SkinSort first checks whether the same patient already has an open case; if so, the new submission is merged with it instead of creating a duplicate. The photo and answers are then sent to the model, which estimates how likely the case is to be routine, priority or urgent. A safety rule comes next: if the chance of the case being urgent is above a deliberately low threshold, it is marked urgent even if another level scored higher. The case is then given a sorting score based on its urgency level, how confident the model is that it is urgent, and when it arrived, and it is placed in the review list. If the model is unavailable for any reason, the case is still saved and simply placed in first-come, first-served order, so nothing is ever lost.

**Stage 3: The dermatologist.** The dermatologist opens the case at the top of the list, looks at the photo and the patient's answers, and makes their own decision. If they agree with the model's urgency level, the case is closed. If they disagree, they set the correct level and that correction is recorded. These corrections are used to monitor how well the model is doing and to retrain it, so SkinSort gets better over time.

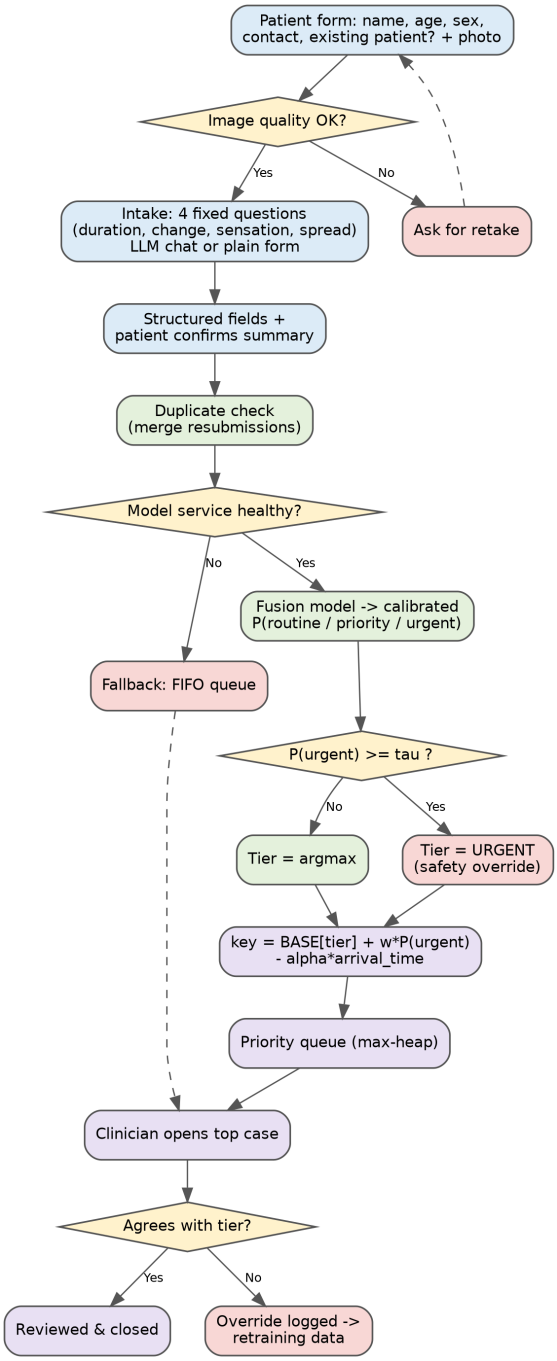



*Figure 3. SkinSort end-to-end flow, from patient upload to dermatologist review*

# 7. Metrics, Objectives, and Business Goals

## 7.1 What We Want to Achieve Once the Model Is Tested

The most important thing for SkinSort is that it does not miss urgent cases. A false alarm only means a harmless spot gets looked at a little sooner; a missed melanoma could mean a cancer goes untreated.

So our main measure is **urgent recall**: out of all the truly urgent cases, how many did the model correctly flag as urgent? Our goal is to catch at least 90 out of every 100 urgent cases on our test data, and at least 85 out of 100 on real phone photos, which are harder.

We also want the model's confidence to be trustworthy. When it says it is 70% sure a case is urgent, it should be right about 70% of the time, because those numbers directly decide the order of the review list. We want the model to perform equally well across skin tones, with no more than a 5-point difference in urgent recall between lighter and darker skin groups. Finally, we expect the model that uses both the photo and the patient's answers to do better than a photo-only version; if it does not, we will report that honestly.

| Measure | Target |
|---|---|
| Urgent cases caught (urgent recall), main test set | 90% or more |
| Urgent cases caught on real phone photos | 85% or more |
| Confidence matches reality (calibration error) | 5% or less |
| Difference in urgent recall across skin tones | 5 points or less |
| Time to score a new case | Under 2 seconds |

### 7.2 Objectives and How They Align With Business Goals

For a clinic, the real goal is simple: get the patients who need care most in front of a dermatologist sooner, without hiring more dermatologists. SkinSort supports this directly. By sorting cases by urgency, it lets the clinic's limited specialist time go where it matters most. We will test this by replaying our test cases through a simulated clinic and comparing SkinSort's order with first-come, first-served. We hope to see urgent cases reviewed at least twice as fast, while no routine case waits longer than a set limit (for example, 30 days).

The project's other objectives also connect to business goals. Keeping a doctor in charge of every decision builds trust and keeps responsibility where it already sits today. Checking fairness across skin tones is not just the right thing to do; in the US, healthcare providers can face legal risk if a tool works worse for some groups. And building the system as a reproducible, monitored pipeline keeps running costs low, makes it easy to update the model, and leaves a clear record of every model version, which is exactly what a hospital or regulator would want to see.

## 8. Failure Analysis

### 8.1 What Could Go Wrong During the Project

The biggest risk while building SkinSort is **fooling ourselves with good-looking results**. If the same photo, or two photos of the same spot, ends up in both the training and test sets, the model will appear far more accurate than it really is. We prevent this by removing duplicates, splitting by person or lesion, and adding an automatic test that fails if any overlap is found.

A second risk is that our training images do not look like what users will send. Most of our images were taken with a professional skin magnifier, while patients will use ordinary phones. The model could perform well in testing and worse in real life. To reduce this, we make training images look more like phone photos and we test specifically on the real phone photos in PAD-UFES-20, reporting any gap openly.

Other risks include the small amount of patient-answer data, urgent cases being rare compared with routine ones, the urgency table being clinically imperfect, cloud costs running higher than expected, and the optional chat assistant eating into time needed for the core system. We handle these with the two-step training approach, extra weight on urgent cases, checking the urgency table against a clinical source, budget alerts on Google Cloud, and treating the chat assistant as a "nice to have" that only gets built once the main pipeline works.

### 8.2 What Could Go Wrong After Deployment

The most serious failure would be the model labelling an urgent case as routine. SkinSort is designed so that even then, the damage is limited: the threshold for "urgent" is set deliberately low, waiting cases keep rising in the list, nothing is ever removed, and a dermatologist still reviews every case. In the worst case, a patient waits about as long as they would have without SkinSort.

Over time, the kind of photos coming in may change, for example because of new phone cameras, different lighting or a different patient population. This is called **drift**, and it can slowly make the model less accurate. Our monitoring (Section 10) watches for it and triggers retraining. There is also the risk that the model becomes less accurate for darker skin, which we guard against by checking fairness before every new model is released. Doctors might also start trusting the score too much, so the score is always shown as a suggestion, and we track how often doctors disagree with it. Blurry photos are caught at upload, repeat submissions from the same patient are merged, and if the chat assistant is used it can only record what the patient said, which the patient then confirms.

### 8.3 When the Pipeline Itself Breaks

Each stage of the pipeline has its own safety net. If a data source changes or a file is corrupted, the data checks stop the pipeline before any training happens. If training fails partway through, it can resume from a saved checkpoint. If a newly trained model turns out worse or less fair than the current one, it is simply not promoted. If a bad update is deployed, the cloud service can roll back to the previous version in minutes. Most importantly, if the model service goes down completely, SkinSort automatically falls back to plain first-come, first-served ordering. Cases keep being collected and no one is lost; the list just stops being sorted by urgency until the model is back.

## 9. Deployment Infrastructure

SkinSort is deployed on Google Cloud Platform (GCP). We chose managed GCP services so the team spends its time on the machine-learning pipeline rather than on looking after servers. Patients and dermatologists only ever need a web browser; everything else runs in the cloud. This section explains the overall architecture, how a live case flows through it, how code and models are safely released, and which platforms are supported.

## 9.1 Overall Architecture

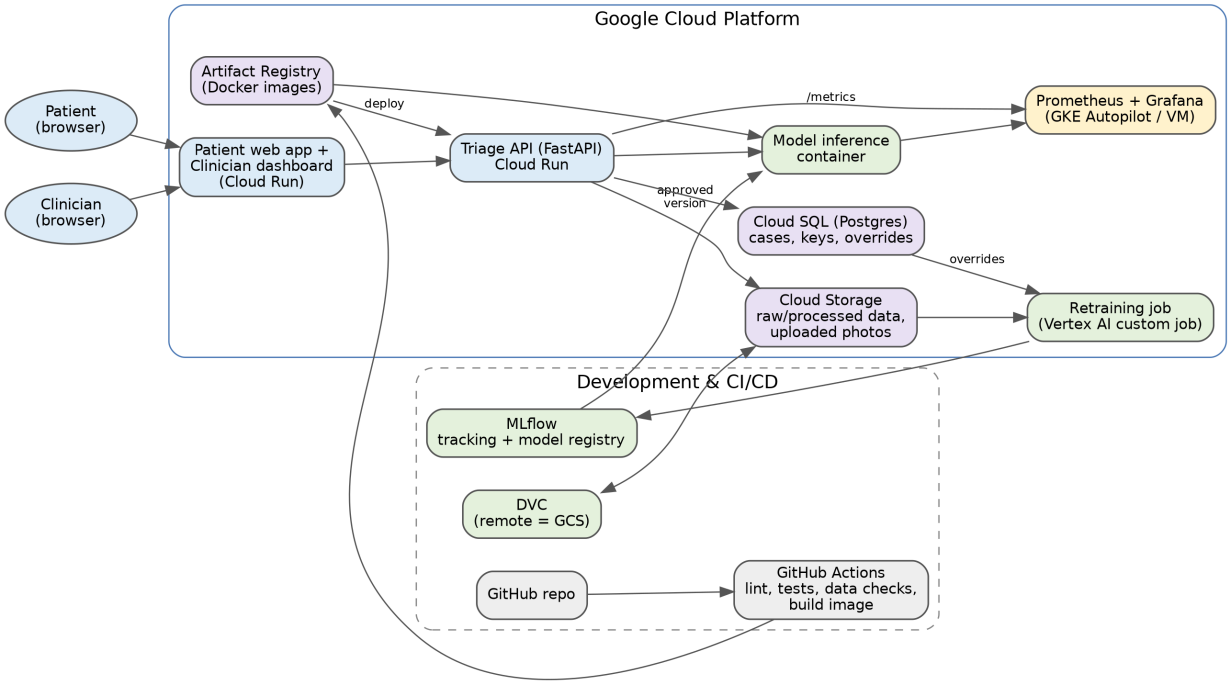



*Figure 4. SkinSort architecture on Google Cloud Platform*

The system is made up of a few building blocks, each with one clear job:

- **Web app (Cloud Run):** the patient upload page and the dermatologist dashboard. Cloud Run automatically adds capacity when traffic rises and scales down to zero when no one is using it, which keeps costs low.
- **Triage API (FastAPI on Cloud Run):** the “front desk” of the system. It receives each new case, saves it, asks the model for a score, calculates the sorting score and hands the next case to the dermatologist. It also exposes measurements for monitoring.
- **Model inference service (Docker container):** runs the trained model and returns urgency probabilities. It is kept separate from the API so the model can be updated without touching the rest of the system.
- **Cloud Storage:** stores uploaded photos as well as all datasets (raw, processed and splits).
- **Cloud SQL (PostgreSQL):** the database that holds each case, its urgency level, its sorting score, the dermatologist’s decision and every correction. Because the review list is stored here, nothing is lost if a service restarts.
- **Vertex AI:** runs training and retraining jobs on GPUs only when needed, so we are not paying for idle machines.
- **MLflow:** records every training run and stores the approved model versions in a model registry.
- **GitHub, GitHub Actions and Artifact Registry:** store the code, automatically test and build it, and keep the packaged Docker images ready to deploy.

- **Prometheus and Grafana:** collect measurements from the running system and show them on dashboards with alerts (see Section 10).

## 9.2 How a Live Case Flows Through the System

When a patient submits a case, the web app sends it to the Triage API. The API stores the photo in Cloud Storage and the case details in Cloud SQL, then asks the model service for a prediction. If the model responds, the API calculates the sorting score and saves it. If the model does not respond, the case is stored in first-come, first-served order instead. Either way, the dermatologist dashboard always shows the highest-priority case first, and the dermatologist's decision is saved back to the database.

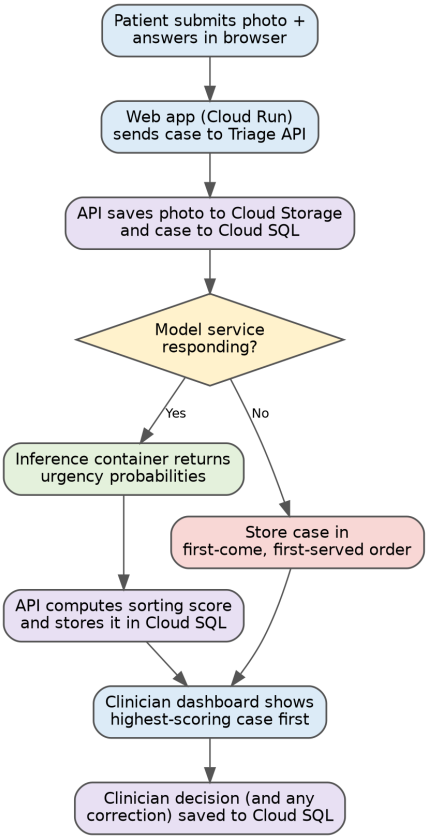



*Figure 5. Flowchart: what happens to a live case*

## 9.3 How Code Is Released (CI/CD)

We never deploy changes by hand. Every code change goes through the same automatic process. When a team member opens a pull request, GitHub Actions runs code style checks, unit tests and data checks. If anything fails, the change cannot be merged. Once merged, the Docker images are built and stored in Artifact Registry, and a new version of the service is deployed to Cloud Run. Before it receives real traffic, a health check and a quick test request confirm it works. If either fails, Cloud Run keeps the previous version running, so a bad release never reaches users.

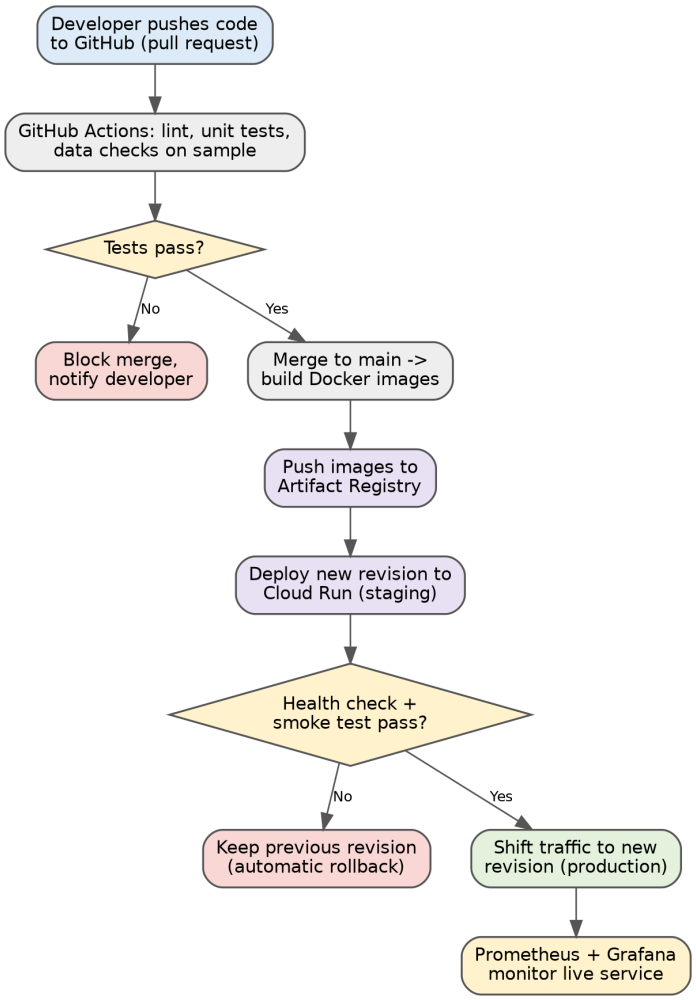



*Figure 6. Flowchart: CI/CD pipeline for code releases*

## 9.4 How a New Model Is Released

New models follow a separate, stricter path. A training job runs on Vertex AI using a specific data version pulled with DVC, and the results are logged to MLflow. The new model is then automatically compared with the current live model on the frozen test sets, including urgent recall, how trustworthy its confidence scores are, and performance across skin tones. If it is not at least as good on every check, it is rejected. If it passes, a team member reviews the results and approves it. Only then is it marked as the production model in MLflow, and the inference service loads the new version.

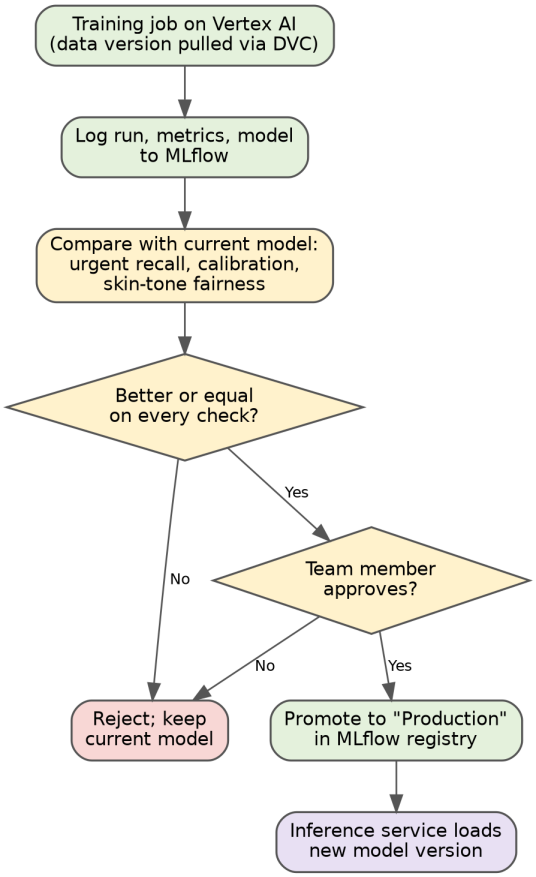



*Figure 7. Flowchart: model release and approval*

### 9.5 Security and Reliability

All traffic uses HTTPS, and photos and data are encrypted automatically in Cloud Storage and Cloud SQL. Each service has its own account with only the permissions it needs, and the dermatologist dashboard requires a login. Every view, score and correction is written to an audit log. For reliability, the review list lives in the database rather than in memory, Cloud Run restarts failed services automatically, and the first-come, first-served fallback keeps cases flowing even if the model is down. A real deployment with patient data would also need a formal HIPAA agreement with Google Cloud.

### 9.6 Supported Platforms

Patients can use SkinSort from any modern web browser on a phone or computer (Chrome, Safari, Firefox or Edge, on iOS, Android, Windows or macOS), so there is no app to install. Dermatologists use the dashboard from a desktop browser. The backend runs in Linux containers on Google Cloud, and developers can run the whole system locally on macOS, Windows or Linux using Docker.

## 10. Monitoring Plan

Once SkinSort is running, we need to know three things at all times: is the system working, is the model still accurate, and is it still fair? Our monitoring plan covers each of these, using Prometheus to collect measurements and Grafana to show them on dashboards and send alerts.

**Is the system working?**

We will track how many cases are coming in, how long each one takes to score, and how often requests fail. Our target is that 95% of cases are scored in under two seconds. If responses slow down or errors start appearing for more than a few minutes, the team gets an alert. We also track every time the system falls back to first-come, first-served ordering, because that means the model is unavailable and urgent cases are no longer being prioritised.

**Are the incoming photos and answers still similar to what we trained on?**

A model is only as good as the data it was trained on. If people start sending photos that are much darker, blurrier or taken with very different cameras, or if the patients are suddenly much older or younger on average, the model may quietly become less accurate. We will continuously compare simple properties of incoming photos (brightness, sharpness, colour) and patient answers with those in our training data. If they drift too far apart, that is a signal to retrain.

**Is the model behaving normally?**

We will watch what share of cases the model labels routine, priority and urgent. If that mix suddenly changes, for example urgent cases jumping from 10% to 40% overnight, something is probably wrong, either with the incoming data or with the model itself. We also watch how often our low “urgent” safety threshold kicks in.

**Do doctors agree with the model?**

In real life we usually do not find out the true diagnosis until much later, so the best live sign of model quality is how often the dermatologist changes the urgency level. A rising disagreement rate tells us the model is getting worse. Since we do not have real dermatologists in this course, these disagreements will be simulated, but the monitoring works exactly the same way.

**Is the model still fair?**

We will break the disagreement rate and accuracy down by skin type, so a drop in performance for any group is noticed early rather than hidden inside an overall average.

**Is anyone waiting too long?**

We will track how many cases are waiting at each urgency level and how long the oldest case has been waiting. This confirms that urgent cases are being reviewed quickly and that routine cases are not being pushed back indefinitely.

**What happens when something goes wrong?**

When the data drifts significantly, the doctor disagreement rate rises noticeably, or a fairness gap appears, a retraining job is started using the newest data and the recorded doctor corrections. The new

model is automatically compared with the current one, including the fairness checks. A team member then reviews the results and approves it before it replaces the live model. Nothing changes in production without a human signing off.

## 11. Success and Acceptance Criteria

We will consider SkinSort successful if it meets three kinds of goals: the model is accurate and fair, the system works reliably from start to finish, and the MLOps practices are solid enough that anyone could rebuild and maintain it.

On the model side, success means catching the large majority of urgent cases (including on real phone photos), giving confidence scores that can be trusted, and performing similarly across skin tones. On the system side, it means a patient can upload a photo and have it scored in under two seconds, the review list sorts correctly, and the system keeps working (in first-come, first-served mode) even when the model is down. In our clinic simulation, urgent cases should be reviewed much faster than with today's first-come, first-served approach, without any routine case being left behind.

On the MLOps side, a new teammate should be able to clone the repository and rebuild the data, model and demo with a few commands. Every deployed model should be traceable to the exact code and data it came from, and the full loop (monitoring spotting a problem, retraining, comparing and approving a new model) should be demonstrated at least once.

| Area | We accept the project when... |
|---|---|
| Model | Urgent recall $\geq$ 90% ($\geq$ 85% on phone photos); skin-tone gap $\leq$ 5 points |
| System | Cases scored in under 2 seconds; fallback mode tested |
| Data | No overlap between training and test data (automated check passes) |
| MLOps | Full pipeline reproducible; monitoring live; retraining loop demonstrated |
| Safety | No score or diagnosis is ever shown to the patient |

## 12. Timeline Planning

The project runs from **September 6, 2026** to our **final presentation on December 8, 2026**. The plan below is preliminary: dates may shift to match course deadlines, feedback from the instructors, and how quickly each stage goes.

| Dates | Phase | Main work |
|---|---|---|
| Sep 6 - Sep 19 | Team formation & ideas | Form the team, explore project ideas, choose the topic |
| Sep 20 - Oct 4 | Scoping & design | Research datasets, design the system, set up GitHub, GCP, DVC and MLflow; submit scoping |
| Oct 5 - Oct 18 | Data pipeline | Download, check, clean and merge data; urgency table; remove duplicates; split data |
| Oct 19 - Nov 1 | First model | Photo-only baseline model; experiment tracking |
| Nov 2 - Nov 15 | Main model & testing | Photo + answers model; confidence calibration; phone-photo and skin-tone fairness tests |
| Nov 16 - Nov 22 | Review list & API | Sorting logic, fallback mode, API, Docker |

| Dates | Phase | Main work |
|---|---|---|
| Nov 23 - Nov 29 | Deployment | Deploy to Google Cloud with automated pipeline; web pages (lighter week for Thanksgiving) |
| Nov 30 - Dec 4 | Monitoring & retraining | Dashboards, alerts, simulated doctor corrections, retraining loop |
| Dec 5 - Dec 7 | Wrap-up | Final testing, clinic simulation results, report and rehearsal |
| Dec 8 | Final presentation | Present SkinSort |

The optional chat assistant for patient questions will only be added if the core system is finished early. Work will be shared across our six team members, covering data, modelling, serving and deployment, monitoring, and the web app, with everyone contributing to testing and the final report.

# 13. Additional Information

## 13.1 Regulation

Because SkinSort is a student prototype that uses public data and is never used on real patients, it does not need FDA approval. If it were ever turned into a real product, it would likely need FDA review, since it influences which patients are seen first. However, it is designed to fall into the lower-risk "decision support" category: it never tells the patient anything, it only reorders a list, and a dermatologist makes every final decision.

## 13.2 The Optional Chat Assistant

If time allows, the four intake questions could be asked through a friendly chat assistant instead of a plain form. Its job would be strictly limited to asking the questions, re-asking when an answer is unclear, and turning the answers into a short summary that the patient confirms. It would never guess what the spot is, reassure or alarm the patient, or influence the urgency score.

## 13.3 Assumptions

Dermatologist corrections are simulated because we do not have real clinicians. Demo patient answers are made up or taken from PAD-UFES-20. Our urgency table is a simplified version of real clinical triage, and the numbers that control list sorting will be tuned during testing.